

Computer Architecture Lab

(EL-2012)

LABORATORY REPORT

Fall 2026



(Lab# **02**)

Implementation of Combinational Circuits using Verilog Gate & Dataflow Level Modeling

Student Name: __Eesha Rehan__ Roll No: __24i-6027__ Section: B__

Date performed: __9th September__, 2026

MARKS AWARDED: ____ / 10

Signature and date of Instructor: _____

Lab Instructors: Dr. Nasir Ali Shah, Engr. Fasih Ahmad

NATIONAL UNIVERSITY OF COMPUTER AND EMERGING SCIENCES, ISLAMABAD

Prepared by: Fasih Ahmad

Version: 1.0

Verified by: Dr. Nasir Ali Shah

Updated: Fall 2026

Task 1(a):

Description:

In this task we write Verilog code by both data flow and gate flow level of the half adder in which we take two inputs as a and b and outputs sum and carry. Firstly, made the circuit on Logisim and test the truth table after that uses the syntax of gate level and data flow level did sum and carry by using gates XOR and AND. Also we made the testbench and tested it and the simulation is giving accurate results comparing to the simulation on Logisim.

Code:

HALFADDER: GATE FLOW LEVEL

```
module halfadder(a,b,s,c);  
  
input a;  
  
input b;  
  
output s;  
  
output c;  
  
xor x1(s,a,b);  
  
and a1(c,a,b);  
  
endmodule  
  
module testbench();  
  
reg a;  
  
reg b;  
  
wire s;  
  
wire c;  
  
halfadder unit(a,b,s,c);  
  
initial  
  
begin
```

```
a=0; b=0;
```

```
#50 a=0; b=1;
```

```
#50 a=1; b=0;
```

```
#50 a=1; b=1;
```

```
end
```

```
endmodule
```

HALFADDER: DATA FLOW LEVEL

```
module halfadder(a,b,s,c);
```

```
input a;
```

```
input b;
```

```
output s;
```

```
output c;
```

```
assign s=a^b;
```

```
assign c=a&b;
```

```
endmodule
```

```
module testbench();
```

```
reg a;
```

```
reg b;
```

```
wire s;
```

```
wire c;
```

```
halfadder unit(a,b,s,c);
```

```
initial
```

```
begin
```

```
a=0; b=0;
```

```
#50 a=0; b=1;
```

```
#50 a=1; b=0;
```

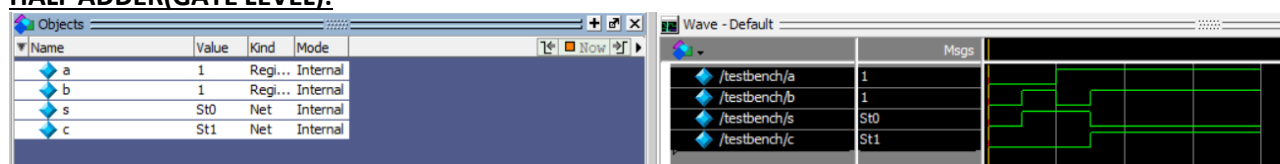
```
#50 a=1; b=1;
```

```
end
```

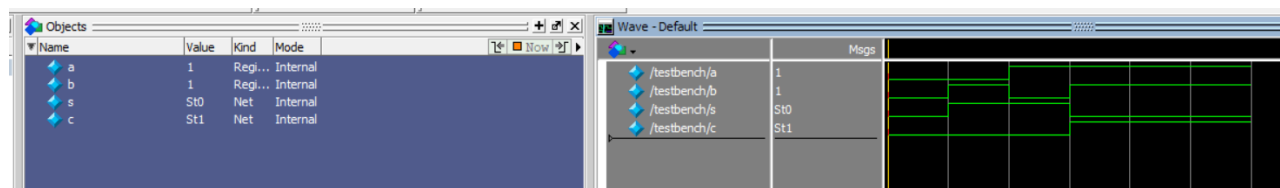
```
endmodule
```

Output:

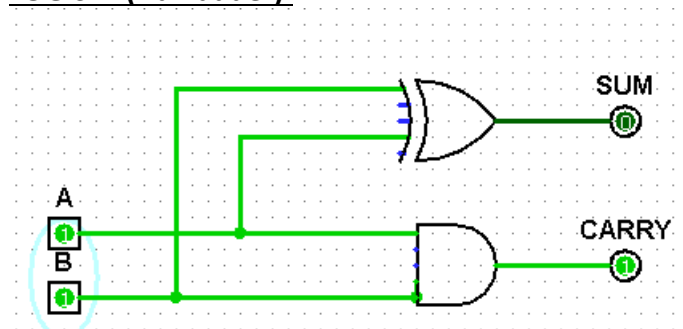
HALF ADDER(GATE LEVEL):



HALF ADDER(DATA FLOW):



LOGISIM(Half adder):



Task 1(b):

Description:

In this task we are required to write Verilog code for full adder by both gate and data flow level so here we use the module of the task 1 in which we made half adder and call that module in full adder module. We take three inputs as a ,b, carry in(cin) and outputs as sum and carry , as we know full adder is made up of two half adders so we call the function of half adder twice and made the test bench after that we get accurate results compared to the circuit on Logisim.

Code:

FULLADDER (gate level):

```
module halfadder(a,b,s,c);  
  
input a;  
  
input b;  
  
output s;  
  
output c;  
  
xor x1(s,a,b);  
  
and a1(c,a,b);  
  
endmodule  
  
module testbench();  
  
reg a;  
  
reg b;  
  
wire s;  
  
wire c;  
  
halfadder unit(a,b,s,c);  
  
initial  
  
begin  
  
    a=0; b=0;  
  
    #50 a=0; b=1;  
  
    #50 a=1; b=0;  
  
    #50 a=1; b=1;  
  
end  
  
endmodule  
  
module fulladder(a,b,cin,s,c);  
  
input a;
```

```
input b;

input cin;

output s;

output c;

wire t1;//sum

wire t2;//carry

wire t3;

halfadder U1(a,b,t1,t2);

halfadder U2(t1,cin,s,t3);

or(c,t2,t3);

endmodule

module testbench();

reg a;

reg b;

reg cin;

wire s;

wire c;

fulladder unit(a,b,cin,s,c);

initial

begin

a=0;b=0;cin=0;

#50 a=0;b=0;cin=1;

#50 a=0;b=1;cin=0;

#50 a=0;b=1;cin=1;
```

```
#50 a=1;b=0;cin=0;

#50 a=1;b=0;cin=1;

#50 a=1;b=1;cin=0;

#50 a=1;b=1;cin=1;

end

endmodule
```

FULLADDER (data flow level):

```
module halfadder(a,b,s,c);

input a;

input b;

output s;

output c;

assign s=a^b;

assign c=a&b;

endmodule

module testbench();

reg a;

reg b;

wire s;

wire c;

halfadder unit(a,b,s,c);

initial

begin

    a=0; b=0;

    #50 a=0; b=1;
```

```
#50 a=1; b=0;

#50 a=1; b=1;

end

endmodule

module fulladder(a,b,cin,s,c);

input a;

input b;

input cin;

output s;

output c;

wire t1;//sum

wire t2;//carry

wire t3;

halfadder U1(a,b,t1,t2);

halfadder U2(t1,cin,s,t3);

assign c=t2|t3;

endmodule

module testbench();

reg a;

reg b;

reg cin;

wire s;

wire c;

fulladder unit(a,b,cin,s,c);
```

```

initial

begin

a=0;b=0;cin=0;

#50 a=0;b=0;cin=1;

#50 a=0;b=1;cin=0;

#50 a=0;b=1;cin=1;

#50 a=1;b=0;cin=0;

#50 a=1;b=0;cin=1;

#50 a=1;b=1;cin=0;

#50 a=1;b=1;cin=1;

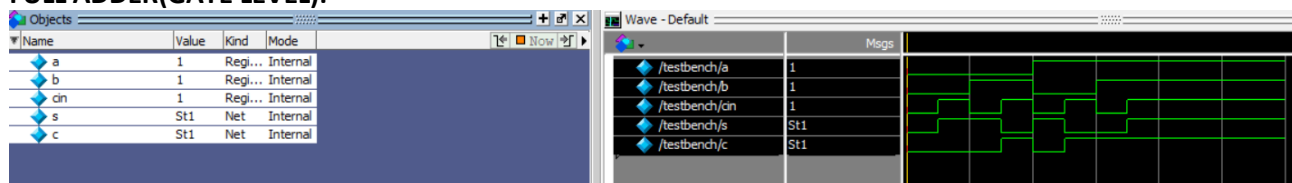
end

endmodule

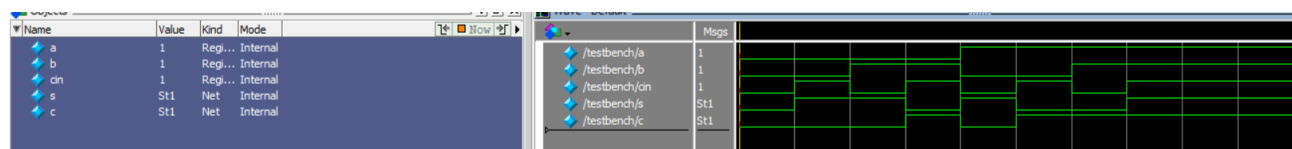
```

Output:

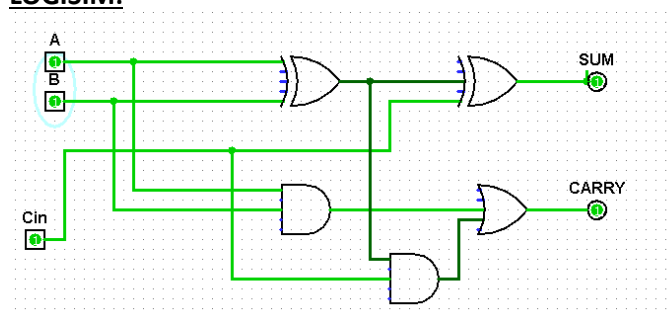
FULL ADDER(GATE LEVEL):



FULL ADDER(DATA FLOW LEVEL):



LOGISIM:



Task 2:

Description:

In this task we are required to make 3-to-8-line decoder in which 3 inputs and 8 outputs are required. on each combination of truth table, it gives 1 on one output and the other are 0. Firstly, we did simulation on Logisim then we write Verilog code by both gate level and data flow level in which we uses not gate and gate and also for storing temporary output we used wires after that we made a test bench in which we tested all 8 combinations of truth table then compile the code and simulate it and compared it with the results shown on Logisim hence they are accurate.

Code:

DECODER(GATE LEVEL)

```
module decoder(a,b,c,y0,y1,y2,y3,y4,y5,y6,y7);  
  
input a;  
  
input b;  
  
input c;  
  
output y0;  
  
output y1;  
  
output y2;  
  
output y3;  
  
output y4;  
  
output y5;  
  
output y6;  
  
output y7;  
  
wire an;  
  
wire bn;  
  
wire cn;  
  
not n1(an,a);
```

```
not n2(bn,b);

not n3(cn,c);

and g0(y0,an,bn,cn);

and g1(y1,an,bn,c);

and g2(y2,an,b,cn);

and g3(y3,an,b,c);

and g4(y4,a,bn,cn);

and g5(y5,a,bn,c);

and g6(y6,a,b,cn);

and g7(y7,a,b,c);

endmodule

module testbench();

reg a;

reg b;

reg c;

wire y0;

wire y1;

wire y2;

wire y3;

wire y4;

wire y5;

wire y6;

wire y7;

decoder unit(a,b,c,y0,y1,y2,y3,y4,y5,y6,y7);
```

```
initial  
  
begin  
  
a=0; b=0; c=0;  
  
#50 a=0; b=0; c=1;  
  
#50 a=0; b=1; c=0;  
  
#50 a=0; b=1; c=1;  
  
#50 a=1; b=0; c=0;  
  
#50 a=1; b=0; c=1;  
  
#50 a=1; b=1; c=0;  
  
#50 a=1; b=1; c=1;  
  
end  
  
endmodule
```

DECODER(DATA FLOW LEVEL):

```
module decoder(a,b,c,y0,y1,y2,y3,y4,y5,y6,y7);  
  
input a;  
  
input b;  
  
input c;  
  
output y0;  
  
output y1;  
  
output y2;  
  
output y3;  
  
output y4;  
  
output y5;  
  
output y6;
```

```
output y7;

wire an;

wire bn;

wire cn;

assign an=~a;

assign bn=~b;

assign cn=~c;

assign y0=an&bn&cn;

assign y1=an&bn&c;

assign y2=an&b&cn;

assign y3=an&b&c;

assign y4=a&bn&cn;

assign y5=a&bn&c;

assign y6=a&b&cn;

assign y7=a&b&c;

endmodule

module testbench();

reg a;

reg b;

reg c;

wire y0;

wire y1;

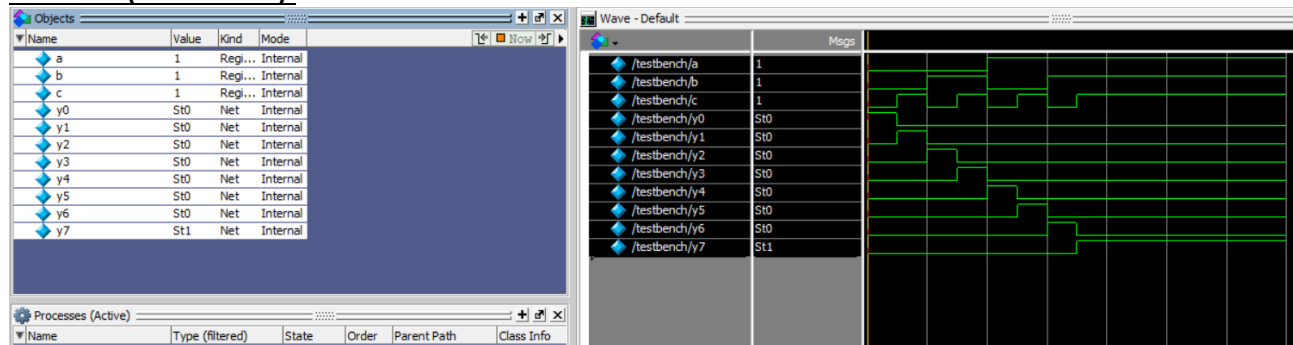
wire y2;

wire y3;
```

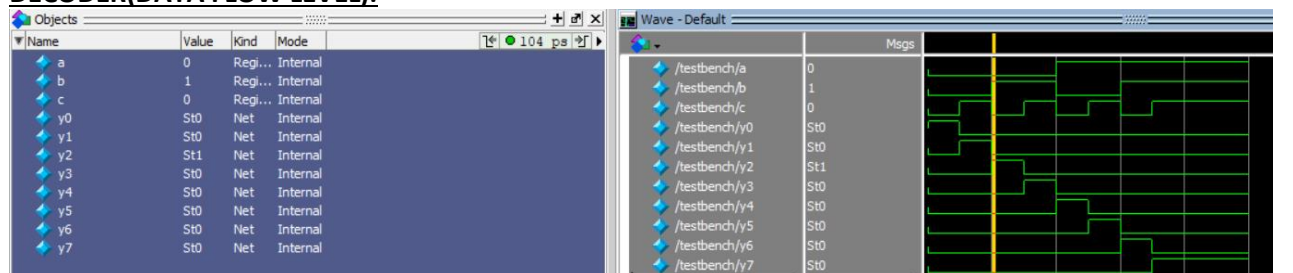
```
wire y4;  
  
wire y5;  
  
wire y6;  
  
wire y7;  
  
decoder unit(a,b,c,y0,y1,y2,y3,y4,y5,y6,y7);  
  
initial  
  
begin  
  
a=0; b=0; c=0;  
  
#50 a=0; b=0; c=1;  
  
#50 a=0; b=1; c=0;  
  
#50 a=0; b=1; c=1;  
  
#50 a=1; b=0; c=0;  
  
#50 a=1; b=0; c=1;  
  
#50 a=1; b=1; c=0;  
  
#50 a=1; b=1; c=1;  
  
end  
  
endmodule
```

Output:

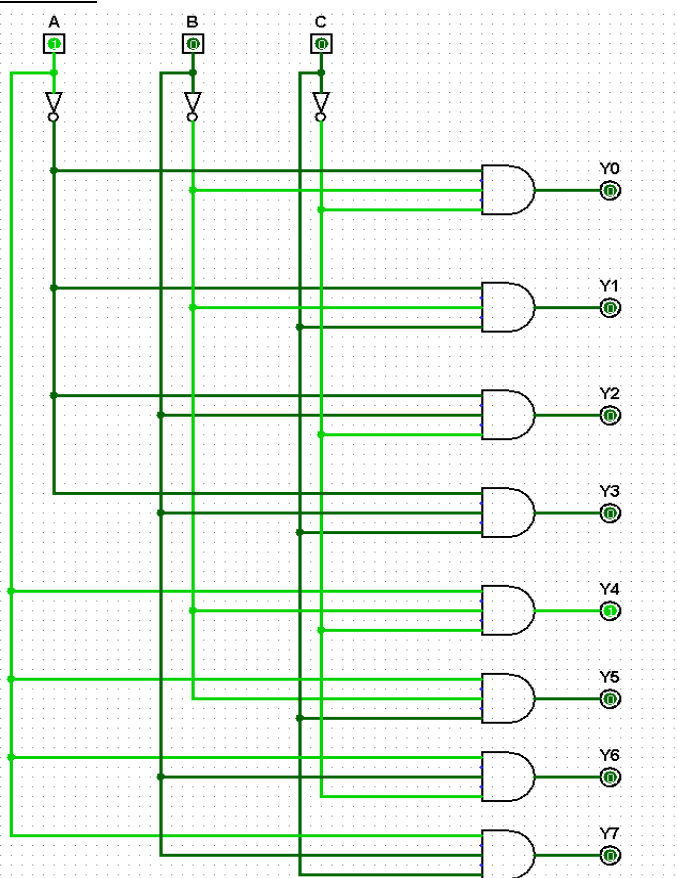
DECODER(GATE LEVEL):



DECODER(DATA FLOW LEVEL):



LOGISIM:



Task 3:

Description:

In this task we required to make mux 8 to 1 line means 8 inputs, 1 output and 3 selection lines. firstly, we did simulation on Logisim in which we take 3 selections lines as input and using the gates or and not . while writing the Verilog code for 8x1 mux we take 8 inputs and 3 selection lines as input and one output. Using wires for storing temporary output and made test bench that will check different combinations of truth table and taken all 8 inputs as 1 so when compiled and it gives 1 output.

Code:

MUX(gate level):

```
module mux(d0,d1,d2,d3,d4,d5,d6,d7,s2,s1,s0,y);  
  
input d0;  
  
input d1;  
  
input d2;  
  
input d3;  
  
input d4;  
  
input d5;  
  
input d6;  
  
input d7;  
  
input s2;  
  
input s1;  
  
input s0;  
  
output y;  
  
wire t2; // NOT s2  
  
wire t1; // NOT s1  
  
wire t0; // NOT s0  
  
wire b0;
```

```
wire b1;

wire b2;

wire b3;

wire b4;

wire b5;

wire b6;

wire b7;

not n1(t2,s2);

not n2(t1,s1);

not n3(t0,s0);

and a0(b0,d0,t2,t1,t0);

and a1(b1,d1,t2,t1,s0);

and a2(b2,d2,t2,s1,t0);

and a3(b3,d3,t2,s1,s0);

and a4(b4,d4,s2,t1,t0);

and a5(b5,d5,s2,t1,s0);

and a6(b6,d6,s2,s1,t0);

and a7(b7,d7,s2,s1,s0);

or o1(y,b0,b1,b2,b3,b4,b5,b6,b7);

endmodule

module testbench();

reg d0;

reg d1;

reg d2;
```

```
reg d3;

reg d4;

reg d5;

reg d6;

reg d7;

reg s2;

reg s1;

reg s0;

wire y;

mux unit(d0,d1,d2,d3,d4,d5,d6,d7,s2,s1,s0,y);

initial

begin

d0=1; d1=1; d2=1; d3=1;

d4=1; d5=1; d6=1; d7=1;

s2=0; s1=0; s0=0;

#50 s2=0; s1=0; s0=1;

#50 s2=0; s1=1; s0=0;

#50 s2=0; s1=1; s0=1;

#50 s2=1; s1=0; s0=0;

#50 s2=1; s1=0; s0=1;

#50 s2=1; s1=1; s0=0;

#50 s2=1; s1=1; s0=1;

end

endmodule
```

MUX(DATA FLOW level):

```
module mux(d0,d1,d2,d3,d4,d5,d6,d7,s2,s1,s0,y);  
  
input d0;  
  
input d1;  
  
input d2;  
  
input d3;  
  
input d4;  
  
input d5;  
  
input d6;  
  
input d7;  
  
input s2;  
  
input s1;  
  
input s0;  
  
output y;  
  
wire t2; // NOT s2  
  
wire t1; // NOT s1  
  
wire t0; // NOT s0  
  
wire b0;  
  
wire b1;  
  
wire b2;  
  
wire b3;  
  
wire b4;  
  
wire b5;  
  
wire b6;
```

```
wire b7;

assign t2=~s2;

assign t1=~s1;

assign t0=~s0;

assign b0=d0&d2&t1&t0;

assign b1=d1&t2&t1&s0;

assign b2=d2&t2&s1&t0;

assign b3=d3&t2&s1&s0;

assign b4=d4&s2&t1&t0;

assign b5=d5&s2&t1&s0;

assign b6=d6&s2&s1&t0;

assign b7=d7&s2&s1&s0;

assign y=b0|b1|b2|b3|b4|b5|b6|b7;

endmodule

module testbench();

reg d0;

reg d1;

reg d2;

reg d3;

reg d4;

reg d5;

reg d6;

reg d7;

reg s2;
```

```

reg s1;

reg s0;

wire y;

mux unit(d0,d1,d2,d3,d4,d5,d6,d7,s2,s1,s0,y);

initial

begin

d0=1; d1=1; d2=1; d3=1;

d4=1; d5=1; d6=1; d7=1;

s2=0; s1=0; s0=0;

#50 s2=0; s1=0; s0=1;

#50 s2=0; s1=1; s0=0;

#50 s2=0; s1=1; s0=1;

#50 s2=1; s1=0; s0=0;

#50 s2=1; s1=0; s0=1;

#50 s2=1; s1=1; s0=0;

#50 s2=1; s1=1; s0=1;

end

endmodule

```

Output:

MUX(GATE LEVEL):

